



Universidad Nacional de San Luis
Facultad de Ciencias Físico Matemáticas y Naturales
Departamento de Informática

Práctico Final Integrador
High Performance Computing en GPU
Generación de Cartoons a partir de Imágenes

Juan Cruz Paez
María Lujan Flores Medina

Profesores:
Fabiana Piccoli
Natalia Miranda

Ingeniería en Informática

2026

Índice general

1. Introducción	1
2. Decisiones de Diseño	3
2.1. Arquitectura en donde se ejecutará el programa	3
2.2. Paradigmas y estrategia general	3
3. Estadísticas y Métricas de Rendimiento	5
3.1. Métricas de Rendimiento	5
3.2. Configuraciones Experimentales	5
3.3. Visualizaciones de Desempeño	6
4. Conclusiones y Análisis de Resultados	9
4.1. Rendimiento de CUDA	9
A. Resultados del Benchmark en Entorno Local (Compu Juan)	10

Capítulo 1

Introducción

El procesamiento digital de imágenes constituye una de las áreas más demandantes en términos computacionales dentro de la informática moderna. Si bien las bases matemáticas de muchas de sus técnicas datan de siglos atrás, su aplicación práctica a escala masiva solo fue posible con el avance tecnológico del hardware de las últimas décadas, que permitió realizar en tiempo razonable la enorme cantidad de operaciones aritméticas involucradas.

Una imagen digital puede representarse como un arreglo bidimensional de píxeles. En el modelo de color **RGB** (Red, Green, Blue) utilizado en este trabajo, cada píxel contiene tres valores enteros en el rango $[0, 255]$, uno por canal de color. Esta representación permite codificar hasta 16,8 millones de tonalidades distintas, superando la capacidad discriminadora del ojo humano. Los casos extremos son el negro $(0, 0, 0)$ y el blanco $(255, 255, 255)$.

El objetivo del presente trabajo es la **generación automática de imágenes tipo *cartoon*** a partir de una imagen original en formato RGB. Esta transformación busca simplificar formas, resaltar contornos y reducir variaciones tonales, logrando una apariencia similar a la de una ilustración o dibujo animado. Para ello se aplican dos procesos independientes sobre la imagen original:

- **Posterización:** reduce el rango total de niveles de color a un conjunto más pequeño de valores discretos. Cada píxel pasa a adoptar el valor del nivel más cercano dentro de ese rango reducido. En imágenes a color se posteriza cada canal por separado y luego se suman las imágenes resultantes.
- **Detección de bordes (borroneado):** pipeline de tres etapas secuencialmente dependientes entre sí:
 1. *Filtrado (blur)*: suaviza la imagen para eliminar ruido.
 2. *Resaltado (highlight)*: amplifica las diferencias entre zonas de borde y no-borde mediante aproximación de gradiente (filtros Prewitt, Sobel u otros).
 3. *Umbralado*: binariza el resultado del resaltado para obtener los bordes detectados.

La imagen cartoon final se obtiene sumando la salida de la posterización y la de la detección de bordes.

Dado el volumen de cómputo involucrado, este trabajo explora y compara distintas **estrategias de paralelización** con el fin de reducir el tiempo de procesamiento. Se implementan las versiones del algoritmo:

- **Secuencial:** implementación de referencia, sin paralelismo.
- **GPU (CUDA):** paralelismo masivo a nivel de *threads* concurrentes en el hardware acelerador, optimizando los accesos mediante una jerarquía de memoria global y compartida.

El análisis de desempeño se realiza sobre imágenes de tres tamaños (800×800 , 2000×2000 y 5000×5000 píxeles), con filtros de convolución de 3×3 y 5×5 , y posterizado configurado con 3 y 9 rangos de color. Las métricas utilizadas son el *speedup* y la eficiencia paralela.

Capítulo 2

Decisiones de Diseño

Arquitectura en donde se ejecutará el programa

El entorno grafico donde se ejecutará el programa es sobre una tarjeta NVIDIA GeForce GTX 1650 utiliza la arquitectura **Turing**.

Limites de la Arquitectura

En una arquitectura de Turing los límites para una grilla de una dimensión (la cual se utilizo en el presente programa) son:

- **Bloques:** El máximo de threads por bloque es de 1024 threads.
- **Grilla:** El tamaño máximo de la grilla es de $2^{31} - 1$ bloques.
- **Tamaño del Warp:** la GPU planifica y ejecuta las instrucciones en grupos de 32 threads.

Paradigmas y estrategia general

Algoritmo secuencial

En esta versión del algoritmo se aplicó la técnica de **padding**: se añade un borde auxiliar de píxeles con valor neutro (negro) alrededor de la matriz de entrada. Esto elimina la necesidad de verificar en cada iteración si las celdas vecinas son válidas al calcular el blur y el highlight, simplificando el código y mejorando la localidad de memoria.

Algoritmo implementado en CUDA

La solución se estructuró de forma modular en tres kernels, acorde a las etapas del proceso (`blurKernel`, `highlightKernel` y `posterizeSumKernel`). A su vez, se unificaron las etapas de posterizado y suma final en un único modulo. Esto garantiza que cada fase opere sobre datos consolidados, simplificando la sincronización de bloques y la reutilización de buferes intermedios en la GPU.

Para la paralelización de los filtros (Blur y Sobel) sobre la GPU utilizando CUDA, se diseñó una estrategia de **descomposición de dominio bidimensional** optimizada

mediante el uso de memoria compartida (*shared memory*) y una zona de **halo** (*halo zone*). A continuación, se describen los pilares de esta estrategia:

- **Mapeo Bidimensional sobre Grilla y Bloques 1D:** En lugar de realizar una paralelización simplista basada en filas o columnas completas (lo cual degradaría la localidad espacial y limitaría el paralelismo en imágenes grandes), la imagen se subdivide en porciones rectangulares o *tiles*. A pesar de que el bloque y la grilla se configuran en una única dimensión (1D) en el código, cada bloque reconstruye su posición bidimensional correspondiente en la grilla (`blockIdx_x` y `blockIdx_y`) utilizando el número de bloques por fila. De manera análoga, cada thread calcula sus coordenadas bidimensionales de píxel (`tx`, `ty`) a partir de su `threadIdx.x` lineal para procesar la porción asignada a su bloque.
- **Estrategia de Halo (Halo Zone) para Filtros de Convolución:** Dado que las operaciones de filtrado por promedio (Blur) y el detector de bordes de Sobel (Highlight) operan sobre una vecindad de píxeles delimitada por un determinado **radio** (1 o 2 píxeles), el bloque necesita acceder a píxeles fuera de su área de procesamiento asignada. Para resolver esta dependencia, se implementa una zona de **halo** periférica en el búfer de memoria compartida. Durante la carga de datos, si los *threads* requieren píxeles que exceden los límites de la imagen, se aplica una condición de borde asignando un valor neutro (color negro: {0, 0, 0}).
- **Uso de Memoria Compartida (*Shared Memory*) y Sincronización:** Para evitar el cuello de botella en el ancho de banda por accesos redundantes a la memoria global de la GPU, se reserva un espacio de memoria compartida (`__shared__ Pixel tile[...]`) para almacenar el fragmento de la imagen y su halo correspondiente. Los *threads* del bloque cooperan para transferir el fragmento de datos desde la memoria global a la compartida. Tras asegurar la carga completa mediante una barrera de sincronización (`__syncthreads()`), los *threads* ejecutan las operaciones de convolución leyendo directamente de la memoria compartida, beneficiándose de una latencia de acceso significativamente menor.

Para la paralelización de el posterizado y la suma, se decidió que la mejor solución es no utilizar la memoria shared y utilizar únicamente la global, esto debido a que el acceso a los datos en estos procedimientos es de una única vez (no se reutilizan los datos a diferencia de Blur y Highlight) lo cual si quisieramos almacenar datos en la memoria shared se deberían hacer más accesos a los datos aparte de que se necesitaría una sincronización entre bloques, lo cual consideramos afectaría negativamente a los tiempos. Otra justificación es que el acceso a la memoria cumple con uno de los requerimientos para aprovechar al máximo la GPU, la cual es el coalescing.

Dado que el radio del filtro de convolución (radio 1 para 3×3 o radio 2 para 5×5) condiona el tamaño del halo como las iteraciones de los bucles, se implementó el kernel mediante plantillas de C++ (`template <int radio>`). Conocer el radio en tiempo de compilación permite al compilador de CUDA calcular de manera estática el tamaño de la memoria compartida.

Capítulo 3

Estadísticas y Métricas de Rendimiento

Se detallan las métricas y estadísticas de rendimiento recopiladas durante las ejecuciones de prueba del generador de imágenes tipo *cartoon*. Se describen los entornos de prueba, las configuraciones experimentales analizadas y se presentan los gráficos resultantes para visualizar el comportamiento de las distintas implementaciones paralelas.

Métricas de Rendimiento

Para evaluar y comparar cuantitativamente el desempeño de las implementaciones paralelas en relación con la versión secuencial de referencia, se utilizaron las siguientes métricas estándar:

- **Tiempo de ejecución (T_p):** El tiempo total medido en segundos empleado en el procesamiento de la imagen. Para garantizar la precisión de la medición del algoritmo en sí, este tiempo excluye las operaciones de entrada/salida (I/O) en disco (carga de la imagen original y guardado del archivo final procesado).
- **Speedup o Aceleración (S_p):** Representa la ganancia de velocidad obtenida al utilizar recursos paralelos en comparación con el algoritmo secuencial. Se define mediante la fórmula:

$$S_p = \frac{T_1}{T_p}$$

Donde T_1 es el tiempo de ejecución secuencial y T_p es el tiempo de ejecución con la versión paralela utilizando p unidades de recursos (threads o procesos). Un speedup ideal sería uno super lineal.

Configuraciones Experimentales

Las pruebas de rendimiento se llevaron a cabo variando múltiples parámetros del algoritmo y del entorno para analizar el comportamiento bajo distintas intensidades de cómputo y comunicación:

1. **Resolución de las Imágenes:** Se utilizaron tres resoluciones con distinta cantidad de carga de trabajo:
 - **Baja** (800×800 **píxeles**): Carga de trabajo ligera (640 mil píxeles).
 - **Media** (2000×2000 **píxeles**): Carga de trabajo moderada (4 millones de píxeles).
 - **Alta** (5000×5000 **píxeles**): Carga de trabajo pesada (25 millones de píxeles).
2. **Intensidad del Filtro de Convolución (Borroneado):**
 - **Filtro 3×3 (Radio 1)**: Requiere menor cantidad de operaciones por píxel.
 - **Filtro 5×5 (Radio 2)**: Requiere mayor cantidad de operaciones de vecindad por píxel.
3. **Rangos de Posterizado:**
 - **3 Rangos (Parámetro 1)**: Pocos intervalos de discretización de color.
 - **9 Rangos (Parámetro 2)**: Mayor granularidad en el cálculo del color final.
4. **Recursos y Paradigmas Asignados:**
 - **Secuencial**: Ejecución base con un solo thread/proceso (1 recurso).
 - **Graphics Processing Unit (GPU)**: evaluado en configuraciones de bloques con *Tiles* de 8×8 , 16×16 , 24×24 y 32×32 , correspondientes a 64, 256, 576 y 1024 threads por bloque.

Visualizaciones de Desempeño

A continuación, se presentan el gráfico generado para representar de manera sintética el comportamiento del sistema.

Heatmap de Speedup por Escenario

El heatmap permite cruzar todas las ejecuciones (combinaciones de resolución, filtro y posterización en el eje X) con todas las configuraciones de recursos (eje Y), mapeando el speedup absoluto en una escala de color.

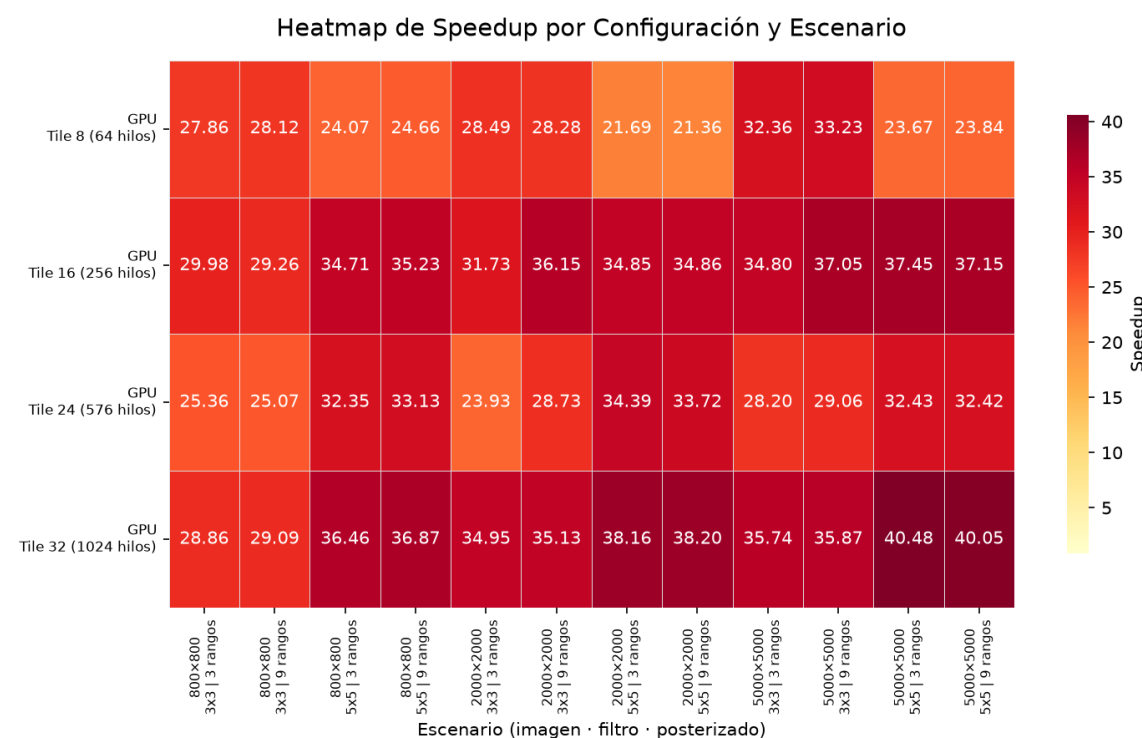


Figura 3.1: Heatmap de Speedup detallado por escenario y configuración.

Análisis de Escalabilidad por Tamaño de Imagen

La gráfica de escalabilidad permite analizar la evolución del *speedup* promedio de las distintas configuraciones de GPU (según el tamaño de (Tile) y números de threads) a medida que incrementa la resolución de la imagen y la carga de cómputo asociada (desde 800×800 hasta 5000×5000 píxeles).

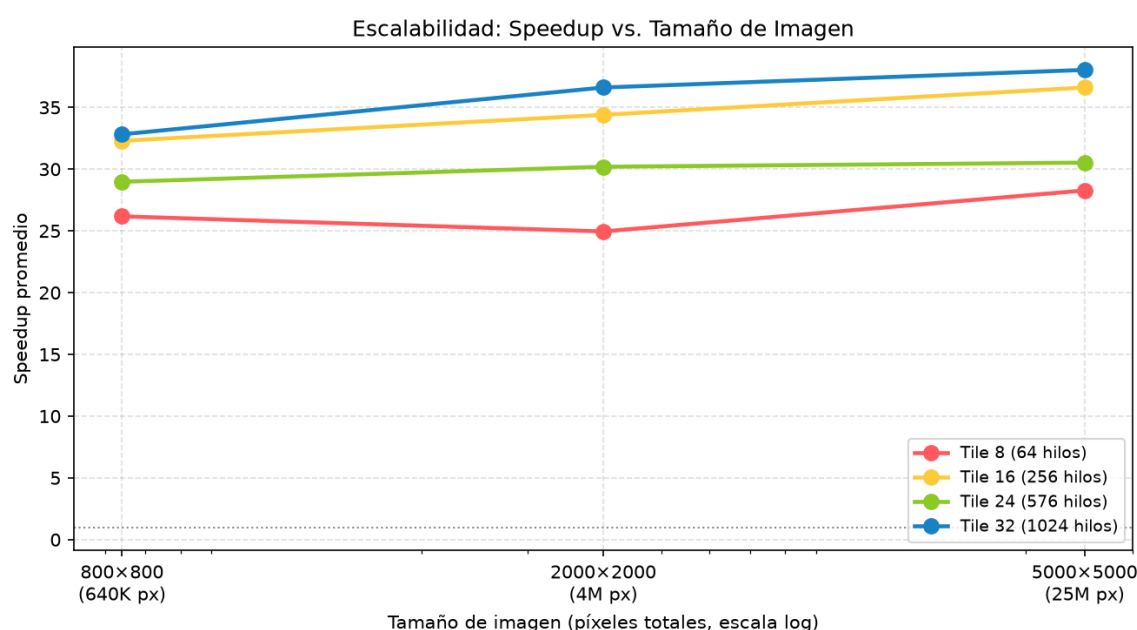


Figura 3.2: Evolución del speedup de la GPU frente al tamaño de la imagen.

Nota: Las tablas numéricas de datos crudos obtenidas de cada ejecución se encuentran adjuntas en los Anexos A.

Capítulo 4

Conclusiones y Análisis de Resultados

Se presenta el análisis de los resultados experimentales obtenidos a partir de las pruebas de rendimiento del generador de *cartoons*. Se contrastan las diferentes implementaciones y se analizan las causas arquitectónicas, de software y de entorno físico que justifican el comportamiento observado en los gráficos.

Rendimiento de CUDA

Se puede ver a simple vista la mejora de rendimiento que se obtiene al implementar en un problema altamente paralelizable como el de el procesamiento de imagenes. El beneficio de la GPU es más pronunciado ante problemas de mayor complejidad computacional. Al pasar de un filtro 3×3 a uno de 5×5 , la versión secuencial duplica ampliamente sus tiempos de ejecución. Sin embargo, la GPU mitiga este sobrecosto gracias al uso de la memoria compartida local (*shared memory*) para almacenar el bloque y la zona de halo, permitiendo que el speedup escale hasta su máximo histórico de **40.48** $\times$ para la imagen de 5000×5000 píxeles. Esto demuestra una excelente escalabilidad a medida que aumenta la carga de trabajo y el volumen de datos.

Apéndice A

Resultados del Benchmark en Entorno Local (Compu Juan)

En este anexo se presentan las tablas de tiempos, speedup y eficiencia obtenidos en el entorno local.

Imagen: 800x800			
Filtro: 3x3, Posterizado: 3 rangos			
Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.022458	1.00x
GPU (Tile 8)	64 hilos	0.000806	27.86x
GPU (Tile 16)	256 hilos	0.000749	29.98x
GPU (Tile 24)	576 hilos	0.000885	25.36x
GPU (Tile 32)	1024 hilos	0.000778	28.86x

Figura A.1: Resultados del benchmark en entorno local para imagen de 800x800, filtro 3x3 y 3 rangos de posterización.

Filtro: 3x3, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.022597	1.00x
GPU (Tile 8)	64 hilos	0.000803	28.12x
GPU (Tile 16)	256 hilos	0.000772	29.26x
GPU (Tile 24)	576 hilos	0.000902	25.07x
GPU (Tile 32)	1024 hilos	0.000777	29.09x

Figura A.2: Resultados del benchmark en entorno local para imagen de 800x800, filtro 3x3 y 9 rangos de posterización.

Filtro: 5x5, Posterizado: 3 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.054859	1.00x
GPU (Tile 8)	64 hilos	0.002279	24.07x
GPU (Tile 16)	256 hilos	0.001580	34.71x
GPU (Tile 24)	576 hilos	0.001696	32.35x
GPU (Tile 32)	1024 hilos	0.001505	36.46x

Figura A.3: Resultados del benchmark en entorno local para imagen de 800x800, filtro 5x5 y 3 rangos de posterización.

Filtro: 5x5, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.055305	1.00x
GPU (Tile 8)	64 hilos	0.002243	24.66x
GPU (Tile 16)	256 hilos	0.001570	35.23x
GPU (Tile 24)	576 hilos	0.001669	33.13x
GPU (Tile 32)	1024 hilos	0.001500	36.87x

Figura A.4: Resultados del benchmark en entorno local para imagen de 800x800, filtro 5x5 y 9 rangos de posterización.

Imagen: 2000x2000

Filtro: 3x3, Posterizado: 3 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.138506	1.00x
GPU (Tile 8)	64 hilos	0.004862	28.49x
GPU (Tile 16)	256 hilos	0.004366	31.73x
GPU (Tile 24)	576 hilos	0.005789	23.93x
GPU (Tile 32)	1024 hilos	0.003963	34.95x

Figura A.5: Resultados del benchmark en entorno local para imagen de 2000x2000, filtro 3x3 y 3 rangos de posterización.

Filtro: 3x3, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.139167	1.00x
GPU (Tile 8)	64 hilos	0.004920	28.28x
GPU (Tile 16)	256 hilos	0.003849	36.15x
GPU (Tile 24)	576 hilos	0.004844	28.73x
GPU (Tile 32)	1024 hilos	0.003961	35.13x

Figura A.6: Resultados del benchmark en entorno local para imagen de 2000x2000, filtro 3x3 y 9 rangos de posterización.

Filtro: 5x5, Posterizado: 3 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.348351	1.00x
GPU (Tile 8)	64 hilos	0.016063	21.69x
GPU (Tile 16)	256 hilos	0.009996	34.85x
GPU (Tile 24)	576 hilos	0.010129	34.39x
GPU (Tile 32)	1024 hilos	0.009128	38.16x

Figura A.7: Resultados del benchmark en entorno local para imagen de 2000x2000, filtro 5x5 y 3 rangos de posterización.

Filtro: 5x5, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.342613	1.00x
GPU (Tile 8)	64 hilos	0.016041	21.36x
GPU (Tile 16)	256 hilos	0.009830	34.86x
GPU (Tile 24)	576 hilos	0.010162	33.72x
GPU (Tile 32)	1024 hilos	0.008968	38.20x

Figura A.8: Resultados del benchmark en entorno local para imagen de 2000x2000, filtro 5x5 y 9 rangos de posterización.

Imagen: 5000x5000

Filtro: 3x3, Posterizado: 3 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.877222	1.00x
GPU (Tile 8)	64 hilos	0.027111	32.36x
GPU (Tile 16)	256 hilos	0.025208	34.80x
GPU (Tile 24)	576 hilos	0.031109	28.20x
GPU (Tile 32)	1024 hilos	0.024545	35.74x

Figura A.9: Resultados del benchmark en entorno local para imagen de 5000x5000, filtro 3x3 y 3 rangos de posterización.

Filtro: 3x3, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.899776	1.00x
GPU (Tile 8)	64 hilos	0.027079	33.23x
GPU (Tile 16)	256 hilos	0.024286	37.05x
GPU (Tile 24)	576 hilos	0.030964	29.06x
GPU (Tile 32)	1024 hilos	0.025088	35.87x

Figura A.10: Resultados del benchmark en entorno local para imagen de 5000x5000, filtro 3x3 y 9 rangos de posterización.

Filtro: 5x5, Posterizado: 3 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	2.154183	1.00x
GPU (Tile 8)	64 hilos	0.091003	23.67x
GPU (Tile 16)	256 hilos	0.057529	37.45x
GPU (Tile 24)	576 hilos	0.066418	32.43x
GPU (Tile 32)	1024 hilos	0.053219	40.48x

Figura A.11: Resultados del benchmark en entorno local para imagen de 5000x5000, filtro 5x5 y 3 rangos de posterización.

Filtro: 5x5, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	2.146076	1.00x
GPU (Tile 8)	64 hilos	0.090005	23.84x
GPU (Tile 16)	256 hilos	0.057761	37.15x
GPU (Tile 24)	576 hilos	0.066201	32.42x
GPU (Tile 32)	1024 hilos	0.053592	40.05x

Figura A.12: Resultados del benchmark en entorno local para imagen de 5000x5000, filtro 5x5 y 9 rangos de posterización.